

The Intelligence Orchestrator.

The canonical specification of how Auxion thinks. It defines an enterprise-grade autonomous business-intelligence orchestration system — a deterministic AI operating system that reasons about a business the way a McKinsey consultant, a Deloitte auditor, a Bain strategist, a Gartner analyst, a Fortune-100 COO, and an AI research scientist would, held to engineering discipline rather than opinion. This is an architecture document. Every section states *why* before *how*. Every subsequent engineering sprint conforms to it.

16 SECTIONS

16 COGNITIVE SUBSYSTEMS

6 LIFECYCLES

DETERMINISTIC

EVIDENCE-FIRST

DOCUMENT STATUS

Supersedes no prior document; governs all future ones. Aligns with PDF 00 (Design System) for language and PDF 26–27 for scan behavior. Where a future document conflicts with AIS-001 on intelligence architecture, AIS-001 prevails.

§ CONTENTS

16 SECTIONS • 6 LIFECYCLES

01	Overall Philosophy	WHY BEFORE HOW	02	Core Principles	15 INVARIANTS
03	The Intelligence Brain	16 SUBSYSTEMS	04	The Thinking Loop	CONTROL PLANE
05	Reasoning Architecture	9 PASSES	06	Evidence System	4 STATES
07	Business Graph	SUBSTRATE	08	Recommendation Engine	DERIVED
09	Competitor Intelligence	NEVER FABRICATED	10	Proposal Engine	8 COMPONENTS
11	Monitoring	7 DETECTORS	12	Memory	RETENTION
13	Model Orchestration	MULTI-MODEL	14	Human Collaboration	GATES
15	Enterprise Readiness	8 CONTROLS	16	Future Evolution	7 HORIZONS

01 OVERALL PHILOSOPHY

WHY AUXION EXISTS

Why Auxion exists

Businesses are advised by consultants who observe partially, reason privately, and cannot be audited or reproduced. Auxion externalizes that expertise into a deterministic system: the same evidence yields the same conclusion, every conclusion is inspectable, and the reasoning survives the person who ran it. It is a business-intelligence operating system, not an assistant.

Why normal AI assistants fail

A general assistant optimizes for a fluent answer. It has no standing evidence base, no confidence accounting, no memory of the account, and no obligation to be reproducible — so it will fabricate a competitor, assert a number it cannot support, and contradict itself between sessions. None of those failures are acceptable when the output is presented to a client as fact.

Why business intelligence needs orchestration

Real analysis is many bounded cognitive acts — discovery, evidence collection, validation, reasoning, critique, recommendation, proposal — each with different reliability and cost. A single model cannot hold all of them to standard at once. Orchestration assigns each act to a specialized subsystem, gates the transitions, and makes the whole process observable and correctable.

02 CORE PRINCIPLES

15 INVARIANTS

These invariants bind every subsystem in this document. A design that satisfies its performance goals but violates one of these is rejected. WHY: business intelligence is presented to clients as fact — the discipline below is what separates it from a plausible-sounding language model.

01 Determinism

Same evidence and same version produce the same conclusion; all nondeterminism is isolated and seeded.

02 Evidence First

No conclusion precedes the evidence that supports it. Collection and classification always come before reasoning.

03 Explainability

Every output exposes the reasoning trace and evidence that produced it. Nothing is a bare assertion.

04 Confidence Before Conclusions

A conclusion is emitted only with a computed confidence; low confidence is reported, not hidden.

05 No Hallucinations

The engine states only what evidence supports. Absence is reported as Unavailable, never filled.

06 Never Fabricate Competitors

The competitive set is discovered from evidence. A missing competitor is stated, never invented.

07 Everything Traceable

Every signal, decision, and override is attributable to a source and an actor.

08 Everything Reproducible

Any past scan can be re-run to the same result from its recorded inputs and versions.

09 Everything Versioned

Prompts, models, weights, and policies are versioned; every output records the versions that made it.

10 Human Override

An operator can inspect, edit, and override any output; the override is logged and attributed.

11 Fail Safely

On failure the engine degrades to a partial, honest result — never a confident wrong one.

12 Cost Awareness

Every scan runs inside a budget; the engine spends tokens and calls against explicit ceilings.

13 Privacy First

Client evidence is isolated, encrypted, and never used to train shared models or leak across tenants.

14 Continuous Learning

The system improves from outcomes and operator corrections without compromising isolation.

15 Business-First Reasoning

Reasoning is anchored in business impact, not technical novelty; every finding maps to a business consequence.

03 THE INTELLIGENCE BRAIN

16 COGNITIVE SUBSYSTEMS

The engine is not one model. It is sixteen independent cognitive subsystems, each with a single responsibility, a defined contract, and its own failure and recovery behavior. WHY: a monolith cannot be tested, versioned, or reasoned about; decomposition makes every part independently verifiable and replaceable. Each subsystem below is specified against ten fields — Purpose, Responsibilities, Inputs, Outputs, Interfaces, Dependencies, Failure Modes, Recovery, Performance Goals, and Future Extensions.

S01 Planner

ORCHESTRATION

PURPOSE	Decide what to investigate and in what order to satisfy the scan objective.	RESPONSIBILITIES	Decompose the objective into questions, sequence them, and declare the evidence each needs.
INPUTS	Scan objective, account profile, current coverage.	OUTPUTS	An ordered plan of questions with required evidence and budget.
INTERFACES	Reads graph and memory; issues tasks on the message bus.	DEPENDENCIES	Business Graph, Memory, Policy Engine.
FAILURE MODES	Over- or under-scoping; planning against stale coverage.	RECOVERY	Supervisor re-plans when coverage or budget assumptions break.
PERF GOALS	Plan produced within seconds; re-plan without full restart.	EXTENSIONS	Learned planning heuristics from prior successful scans.

S02 Discovery

SURFACE

PURPOSE	Resolve a URL and identifiers into a bounded, crawlable surface.	RESPONSIBILITIES	Enumerate routes, sitemaps, and connected public properties; set the scan boundary.
INPUTS	Root URL, public identifiers, plan.	OUTPUTS	A scoped surface manifest for the crawler.
INTERFACES	Emits manifest to Crawler; records boundary in graph.	DEPENDENCIES	Planner, external DNS/HTTP.
FAILURE MODES	Unreachable host; ambiguous or hostile surface.	RECOVERY	Retry with backoff; degrade to a partial boundary and flag it.
PERF GOALS	Boundary resolved under a fixed ceiling; polite to targets.	EXTENSIONS	Deeper property discovery via authorized connectors.

S03 Crawler

ACQUISITION

PURPOSE	Fetch pages and assets across the surface under budget and rate limits.	RESPONSIBILITIES	Render where needed, capture raw signals, respect robots and rate policy.
INPUTS	Surface manifest, crawl budget.	OUTPUTS	Raw captured signals with fetch metadata.
INTERFACES	Streams captures to Evidence Engine.	DEPENDENCIES	Discovery, Policy Engine, rate limiter.
FAILURE MODES	Timeouts, blocks, throttling, render failures.	RECOVERY	Per-fetch retry; skip and mark Unavailable past the ceiling.
PERF GOALS	Bounded per-page latency; parallel within limits.	EXTENSIONS	Headless render pools and authenticated fetch.

S04 Evidence Engine

CLASSIFICATION

PURPOSE	Turn raw captures into classified, weighted, provenanced evidence.	RESPONSIBILITIES	Extract signals; assign Observed/Estimated/Inferred/Unavailable; weight, timestamp, and record lineage.
INPUTS	Raw captures, extraction rules.	OUTPUTS	Evidence records with state, weight, freshness, and source.
INTERFACES	Writes evidence into the Business Graph.	DEPENDENCIES	Crawler, Business Graph.
FAILURE MODES	Misclassification; extraction drift; contradictory signals.	RECOVERY	Validator flags contradictions; low-weight items re-collected.
PERF GOALS	High extraction precision; classification is auditable.	EXTENSIONS	New extractors per evidence domain.

S05

Business Graph

SUBSTRATE

PURPOSE	Hold the normalized, relational, temporal model of the business and market.	RESPONSIBILITIES	Store entities, relationships, versions, and provenance; serve queries to reasoning.
INPUTS	Evidence records, prior snapshots.	OUTPUTS	A queryable graph with temporal and provenance edges.
INTERFACES	Read/write API for all reasoning subsystems.	DEPENDENCIES	Evidence Engine, Memory.
FAILURE MODES	Inconsistency; unbounded growth; stale edges.	RECOVERY	Versioned writes; reconciliation jobs; snapshot rollback.
PERF GOALS	Low-latency traversal at account scale.	EXTENSIONS	Cross-account benchmark graph (isolated).

S06

Reasoning Engine

COGNITION

PURPOSE	Produce findings, dimension scores, and the Business Health Index from the graph.	RESPONSIBILITIES	Run the multi-pass reasoning strategy; attach evidence and confidence to every finding.
INPUTS	Graph queries, benchmarks, plan.	OUTPUTS	Findings and scores with confidence and evidence links.
INTERFACES	Consumes graph; emits findings to Recommendation Engine.	DEPENDENCIES	Business Graph, Model Orchestration, Critic.
FAILURE MODES	Overconfident or unsupported findings; reasoning loops.	RECOVERY	Critic rejects; re-reason with tighter evidence scope.
PERF GOALS	Findings within budget; bounded pass count.	EXTENSIONS	Domain-specific reasoning modules.

S07

Recommendation Engine

DERIVATION

PURPOSE	Derive scored, evidence-linked moves from validated findings.	RESPONSIBILITIES	Compute ROI, risk, priority, impact, difficulty, dependencies; assign tiers.
INPUTS	Validated findings, cost model.	OUTPUTS	Ranked recommendations with full attribute sets.
INTERFACES	Emits to Proposal Engine and the surface.	DEPENDENCIES	Reasoning Engine, Policy Engine.
FAILURE MODES	Miscomputed ROI; unstable ranking.	RECOVERY	Recompute from findings; validator bounds estimates.
PERF GOALS	Deterministic ranking; stable across re-runs.	EXTENSIONS	Portfolio optimization across moves.

S08 Proposal Generator

PACKAGING

PURPOSE	Assemble approved recommendations into a client-ready proposal.	RESPONSIBILITIES	Build timeline, pricing, deliverables, milestones, scope, assumptions, risks, ROI.
INPUTS	Approved recommendations, pricing rules.	OUTPUTS	A structured proposal inheriting the evidence trace.
INTERFACES	Renders to the report surface and export.	DEPENDENCIES	Recommendation Engine, Human Collaboration.
FAILURE MODES	Scope drift; pricing inconsistency.	RECOVERY	Regenerate from approved set; human review gate.
PERF GOALS	Proposal assembled deterministically from inputs.	EXTENSIONS	Template variants per engagement type.

S09 Monitoring Engine

CONTINUITY

PURPOSE	Keep an account current after the first scan.	RESPONSIBILITIES	Schedule re-scans, detect change, trend, and raise prioritized alerts.
INPUTS	Schedule, prior graph, new evidence.	OUTPUTS	Change sets, trends, and alerts.
INTERFACES	Feeds Supervisor and the surface.	DEPENDENCIES	Business Graph, Memory, Evidence Engine.
FAILURE MODES	Alert fatigue; missed regressions.	RECOVERY	Threshold tuning; anomaly re-check before alerting.
PERF GOALS	Timely detection within the scan cadence.	EXTENSIONS	Predictive alerts ahead of change.

S10 Memory

PERSISTENCE

PURPOSE	Persist what must be remembered and expire what must not linger.	RESPONSIBILITIES	Store snapshots, trends, and learned heuristics under retention and isolation policy.
INPUTS	Graph snapshots, outcomes, corrections.	OUTPUTS	Durable, queryable historical state.
INTERFACES	Read/write for Planner, Reasoning, Monitoring.	DEPENDENCIES	Business Graph, Policy Engine.
FAILURE MODES	Leakage across tenants; unbounded retention.	RECOVERY	Isolation enforcement; TTL expiry; audit.
PERF GOALS	Fast snapshot read; policy-bound retention.	EXTENSIONS	Cross-engagement learning within isolation.

S11 Policy Engine

GOVERNANCE

PURPOSE	Enforce the rules of the road for every action.	RESPONSIBILITIES	Apply budget, rate, privacy, and compliance policy at each transition.
INPUTS	Action requests, tenant policy.	OUTPUTS	Allow/deny decisions with reasons.
INTERFACES	Gates the message bus.	DEPENDENCIES	All subsystems.
FAILURE MODES	Over-blocking; policy gaps.	RECOVERY	Fail-closed on ambiguity; policy versioning.
PERF GOALS	Negligible per-decision latency.	EXTENSIONS	Per-tenant custom policy packs.

S12 Validator

VERIFICATION

PURPOSE	Check artifacts against the evidence and the invariants.	RESPONSIBILITIES	Verify citations, ranges, contradictions, and schema before an artifact advances.
INPUTS	Any produced artifact, evidence.	OUTPUTS	Validation verdict with violations.
INTERFACES	Gates transitions between subsystems.	DEPENDENCIES	Business Graph, Policy Engine.
FAILURE MODES	False pass; false reject.	RECOVERY	Escalate to Critic; block on hard-invariant breach.
PERF GOALS	High recall on invariant violations.	EXTENSIONS	Learned validators from past defects.

S13 Critic

ADVERSARY

PURPOSE	Argue against the engine's own conclusions.	RESPONSIBILITIES	Generate counter-arguments and alternative hypotheses; stress findings for weakness.
INPUTS	Findings and recommendations.	OUTPUTS	Critiques, counter-evidence requests.
INTERFACES	Feeds Reasoning and Validator.	DEPENDENCIES	Reasoning Engine, Model Orchestration.
FAILURE MODES	Over-critique stalling progress.	RECOVERY	Supervisor bounds critique rounds.
PERF GOALS	Meaningful critique within pass budget.	EXTENSIONS	Domain-expert critic personas.

S14 Supervisor

CONTROL

PURPOSE	Drive the thinking loop and hold global state.	RESPONSIBILITIES	Sequence subsystems, manage retries, enforce budgets, decide completion.
INPUTS	Plan, subsystem signals, budget.	OUTPUTS	Control decisions and run status.
INTERFACES	Owns the control plane on the bus.	DEPENDENCIES	Every subsystem.
FAILURE MODES	Deadlock; runaway loops.	RECOVERY	Timeouts, circuit breakers, safe halt.
PERF GOALS	Low control overhead; predictable termination.	EXTENSIONS	Multi-scan orchestration.

S15 Reflection Layer

INTROSPECTION

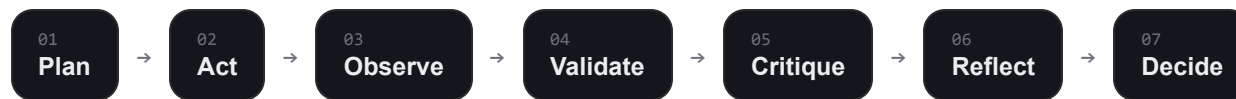
PURPOSE	Assess how well a completed run performed.	RESPONSIBILITIES	Score coverage, confidence, and cost; record what to do differently.
INPUTS	Completed run trace, outcomes.	OUTPUTS	Reflection report and tuning signals.
INTERFACES	Feeds Memory and Continuous Learning.	DEPENDENCIES	Memory, Supervisor.
FAILURE MODES	Misattributed cause; noisy signal.	RECOVERY	Aggregate over runs before acting.
PERF GOALS	Reflection produced post-run cheaply.	EXTENSIONS	Automated strategy adjustment.

S16 Continuous Learning

ADAPTATION

PURPOSE	Improve the engine from outcomes and operator corrections.	RESPONSIBILITIES	Update heuristics, weights, and prompts under versioning and isolation.
INPUTS	Reflection signals, corrections, outcomes.	OUTPUTS	New versioned strategies and weights.
INTERFACES	Publishes versions to all subsystems.	DEPENDENCIES	Reflection Layer, Policy Engine.
FAILURE MODES	Regression; overfitting to one account.	RECOVERY	Shadow-test before promotion; rollback on regression.
PERF GOALS	Safe, measurable improvement over time.	EXTENSIONS	Closed-loop optimization with guardrails.

WHY: subsystems must coordinate without coupling. The Supervisor runs a closed control loop; every subsystem communicates through a typed message bus carrying an envelope of payload, confidence, and evidence lineage. HOW: the loop below repeats until the plan is satisfied or a policy gate halts it.



Uncertainty propagation

Each artifact carries confidence; a low-confidence input caps the confidence of anything derived from it, so weakness never launders into certainty downstream.

Confidence propagation

Corroboration raises confidence along evidence edges; the graph recomputes dependent scores when new evidence lands.

Failure propagation

A failed stage emits a typed fault, not silence. The Supervisor retries, degrades, or halts, and the fault is recorded in the run trace.

05 REASONING ARCHITECTURE

9 PASSES

WHY: a single forward pass produces confident but unverified claims. Auxion reasons in multiple passes, generating alternatives and its own counter-arguments before committing. Each pass hands a checked artifact to the next.

01

Chain of Reasoning

Decompose the question into an explicit, inspectable chain of steps rather than a single leap.

02

Multi-Pass Reasoning

Run the chain more than once with varied framing to expose instability in the conclusion.

03

Alternative Hypotheses

Generate competing explanations for the same evidence instead of committing to the first.

04

Counter-Argument

The Critic argues against each leading hypothesis to find where it breaks.

05

Self-Consistency

Compare passes and hypotheses; a conclusion must survive agreement across them.

06

Verification

The Validator checks the surviving conclusion against evidence, ranges, and invariants.

07	Reflection	Assess the quality and coverage of the reasoning before it is allowed to stand.
08	Critique Resolution	Resolve outstanding critiques by collecting evidence or lowering confidence — never by ignoring them.
09	Final Decision	Commit the conclusion with its confidence and full evidence trace, or report it as unresolved.

06 EVIDENCE SYSTEM

4 STATES · FULL LINEAGE

WHY: every conclusion is only as trustworthy as the evidence beneath it. Each signal enters classified into one of four states, weighted, aged, and carrying complete provenance. The state and lineage travel through every layer to the client-facing report.

● Observed · directly measured ● Estimated · modeled from data ● Inferred · reasoned, not seen
● Unavailable · not yet accessible

Weighted evidence	Each record carries a weight from source reliability and directness; reasoning sums weight, not count.	Contradictions	Conflicting signals are surfaced and reconciled explicitly, never silently averaged away.
Evidence aging	Every record has an age; the engine knows how old each fact is at the moment it reasons.	Evidence decay	Reliability decays past a per-dimension useful window, lowering weight until refreshed.
Evidence confidence	A per-record confidence feeds the composite confidence of any conclusion built on it.	Evidence freshness	Freshness is tracked separately from age and gates whether a fact may drive a live score.
Evidence lineage	Every record links to the exact reasoning and outputs it influenced, in both directions.	Evidence provenance	Every record names its source, capture method, and timestamp — inspectable in the report.

07 BUSINESS GRAPH

RELATIONAL SUBSTRATE

WHY A GRAPH, NOT TABLES

Business intelligence is about relationships — a risk in one capability propagates to another; a competitor advantage maps to a client gap; a signal today revises a conclusion from last quarter. Tables force these relationships into joins that reasoning cannot traverse cheaply. A graph makes propagation, temporality, and provenance first-class and queryable.

Knowledge relationships

Entities and facts connect directly, so reasoning traverses meaning instead of reconstructing it from joins.

Temporal relationships

Every edge is versioned in time; the graph answers not just what is true but when it became true.

Competitor relationships

Competitors link to the client on shared dimensions, making every comparison like-for-like.

Risk propagation

A risk edge carries exposure from one capability to the others it endangers.

Opportunity propagation

An opportunity in one dimension surfaces the adjacent dimensions it unlocks.

Business capability graph

The business is modeled as capabilities and their dependencies, not a flat list of scores.

08 RECOMMENDATION ENGINE

DERIVED, NOT WRITTEN

FIRST PRINCIPLE

Recommendations are never written directly by a language model. They are **derived mathematically** from validated findings in the graph — a model may phrase a recommendation, but its existence, priority, and scores come from computation, not composition.

Born from findings

A recommendation exists only because a validated finding requires it — never authored free-hand.

ROI estimation

Projected return computed from the finding's impact and the cost model, with a confidence band.

Risk estimation

The exposure of not acting, and the risk introduced by acting, both scored.

Priority

A deterministic function of impact, risk, effort, and dependency order.

Impact

The modeled movement in the affected Index dimensions if the move is executed.

Confidence

Inherited from the evidence and findings the recommendation is derived from.

Dependencies

Explicit prerequisites among moves, enforced when they are sequenced into a plan.

Quick wins

Low-effort, fast-impact moves surfaced first for engagement momentum.

Strategic initiatives

High-effort, structural moves that change trajectory over a quarter or more.

09 COMPETITOR INTELLIGENCE

NEVER FABRICATED

WHY: a fabricated competitor destroys trust instantly. The engine discovers a real competitive set from evidence, scores similarity, and handles both false positives and missing competitors explicitly rather than inventing to fill the grid.

Discovery

Identify a candidate set from market, category, and observed positioning in the evidence.

Selection

Keep only competitors with sufficient evidentiary overlap; the rest are dropped, not shown.

Similarity scoring

Score each candidate's relevance to the client's actual market and offering.

Gap analysis

Measure where the client leads or trails on each shared dimension, with evidence.

Benchmarking

Position the client against the set and against category baselines.

False-positive handling

A candidate that fails the evidence threshold is removed rather than kept to pad the grid.

Missing-competitor handling

When a known competitor cannot be scanned, it is reported Unavailable, never estimated into existence.

10 PROPOSAL ENGINE

8 COMPONENTS

WHY: a recommendation is a finding; a proposal is a commitment. The engine assembles approved recommendations into a consulting-grade proposal, inheriting the evidence trace so every line is defensible in front of a client.

Pricing Rolled from each approved move's estimated cost against pricing rules.	Timeline Sequenced phases derived from move dependencies and tier.	Deliverables One concrete output per move, with acceptance criteria.	Milestones Checkpoints tied to measurable movement in the Index.
Scope Exactly the approved moves — nothing outside the evidence trace.	Assumptions Every assumption stated explicitly and linked to its evidence gap.	Risks Delivery and business risks carried forward from the recommendations.	Expected ROI Aggregated return across moves, with the confidence band intact.

11 MONITORING

7 DETECTORS

WHY: a business is a moving target. Once live, the engine scans continuously and distinguishes signal from noise — a genuine regression, an anomaly, or a new opportunity — before it spends an operator's attention on an alert.

Continuous scanning	Re-scan on a per-account cadence, reusing fresh cached evidence.	Change detection	Diff new evidence against the prior graph to isolate what actually moved.
Anomaly detection	Flag values that deviate from the account's own established pattern.	Drift	Detect slow, directional movement that no single scan would surface.
Regression	Catch a dimension or Index falling below a prior established level.	Opportunity detection	Surface newly opened gaps against competitors or benchmarks.

Alert priorities

Rank alerts by impact and confidence so operator attention goes to what matters.

12 MEMORY

RETENTION MODEL

WHY: intelligence compounds only if it remembers. Memory is explicit about what persists, what expires, and what is learned across accounts — never leaking one client's evidence into another's reasoning.

What is remembered

Graph snapshots, dimension trends, outcomes, and operator corrections per account.

What expires

Raw captures and transient working state expire on policy TTL once distilled into evidence.

Snapshots

Immutable point-in-time graph states enabling exact reproduction of any past scan.

Historical trends

Per-dimension and Index history retained to power trend and regression detection.

Long-term learning

Anonymized heuristics improve the engine without moving any client's evidence across tenants.

13 MODEL ORCHESTRATION

NO VENDOR LOCK-IN

WHY: no single model is best at every task, and no vendor should be load-bearing. Models sit behind one internal interface. For high-stakes conclusions the engine runs several in parallel and resolves consensus; for routine work it picks the cheapest model that meets the quality bar.

Why multiple models

No model leads on extraction, deep reasoning, and writing at once; the engine matches model to task.

Model selection

Chosen per task on quality, latency, cost, context size, and live availability.

Parallel reasoning

High-stakes conclusions run across several models concurrently.

Consensus

Agreement across models raises confidence; the composite reflects it.

Conflict resolution

Disagreement triggers more evidence or a lowered confidence — never an arbitrary pick.

Cost optimization

Routine tasks route to the cheapest model that clears the quality bar.

Fallback strategy

An ordered alternate set takes over on provider failure without changing outputs.

14 HUMAN COLLABORATION

APPROVAL GATES

WHY: the engine advises; humans decide. Every output passes through review and approval gates before it reaches a client, and any output remains editable with the override logged.

Consultant review

An operator inspects findings and recommendations with full evidence before anything advances.

Client review

The client sees only reviewed, approved output, with evidence states shown honestly.

Approval gates

Defined checkpoints where a human must approve before an artifact proceeds.

Override logic

Any output can be overridden; the override, actor, and reason are logged.

Editable outputs

Every artifact is editable, and edits are versioned alongside the generated original.

15 ENTERPRISE READINESS

8 CONTROLS

WHY: the engine handles clients' most sensitive business data at scale. Isolation, auditability, and compliance are architectural, not bolted on.

Scaling

Stateless workers scale horizontally; scans run independently and in parallel.

Multi-tenancy

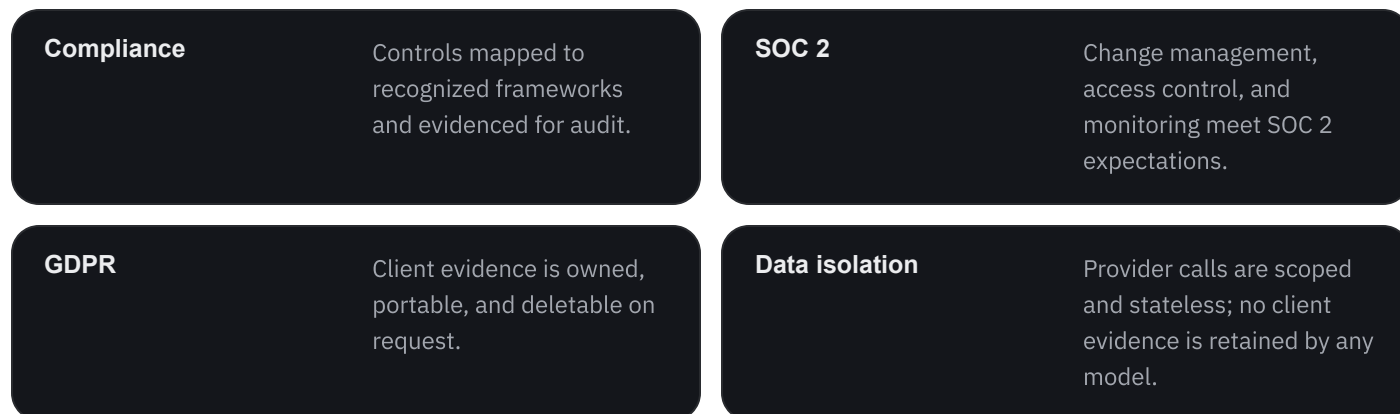
Every tenant is isolated end to end; no scan can read another tenant's data.

Security

Evidence encrypted in transit and at rest; secrets in a vault, never in code or logs.

Audit logs

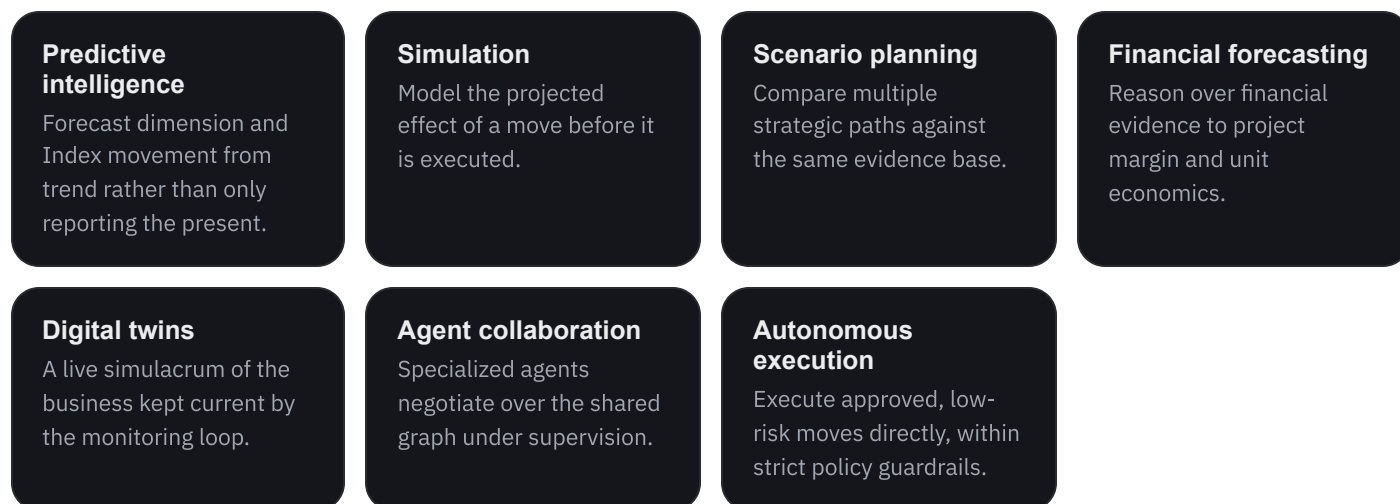
Every access, decision, and override is logged and attributable to an actor.



16 FUTURE EVOLUTION

7 HORIZONS

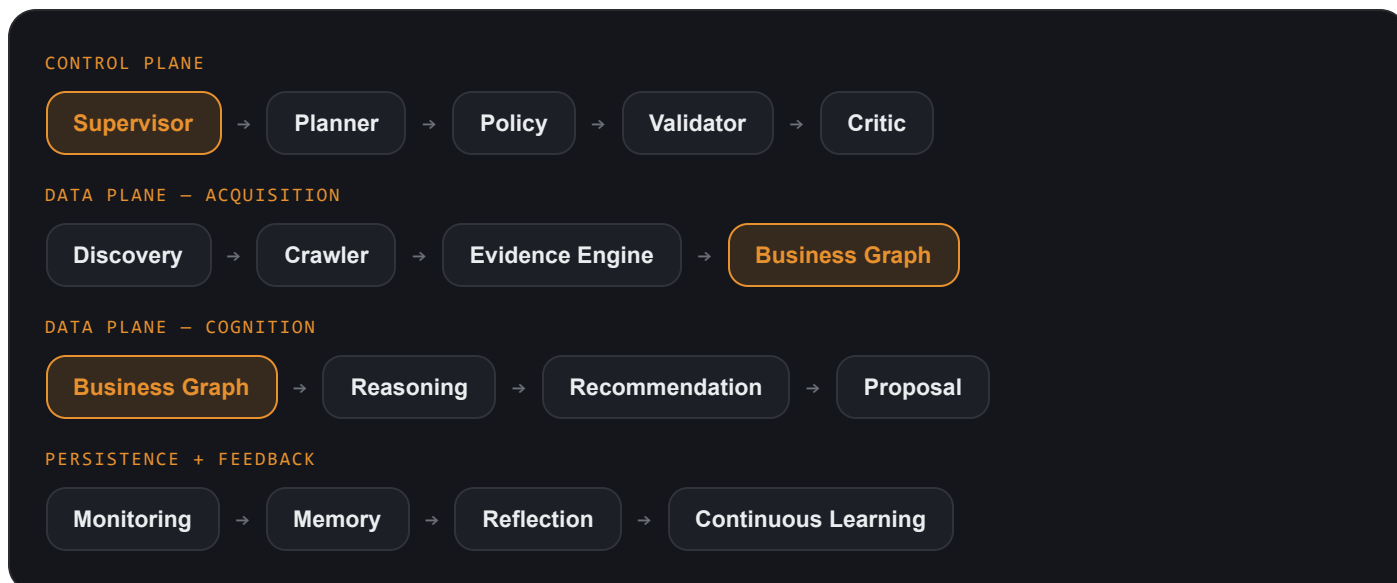
WHY: the architecture must accommodate what is coming without a rewrite. Each horizon below is a new capability plugged into the existing brain, evidence system, and graph — not a new engine.



A SYSTEM INTERACTION DIAGRAM

CONTROL + DATA PLANE

The Supervisor drives the control plane; subsystems exchange typed envelopes on the data plane. Policy, Validator, and Critic gate every transition; Memory and the Business Graph are read and written throughout.



B LIFECYCLES

6 END-TO-END FLOWS

Each lifecycle is the ordered path an artifact takes across the subsystems, from trigger to durable outcome.

L1 Request lifecycle

Trigger → Supervisor → Plan → Policy gate → Execute → Validate → Deliver → Log

L2 Intelligence lifecycle

Objective → Plan → Reason → Critique → Validate → Score → Findings

L3 Evidence lifecycle

Capture → Classify → Weight → Graph → Age / decay → Refresh → Retire

L4 Recommendation lifecycle

Finding → Derive → Score → Rank → Human review → Approve

L5 Proposal lifecycle

Approved moves → Assemble → Price → Review → Sign-off → Export

L6 Monitoring lifecycle

Schedule → Re-scan → Diff → Detect → Prioritize → Alert → Update